Sol Quick Reference

Quick reference for ASU's Sol supercomputer - the solx CLI, Slurm job routing, safe automation, storage, and compute-node access.

v1.0.3

>_

ACCOUNT / ACCESS

Your account decides which partitions and QOS you may use. Check it before choosing a route:

```
sacctmgr -n show assoc user=$USER format=Account,Partition,QOS
myfairshare    # read RealFairShare; a very low score means longer queue waits
```

ROUTING / PARTITIONS

GPUs live in `htc`, `public`, `general`, `lightwork`, and `arm`. The deciding questions are how long the job runs, whether it may be preempted, and which accelerator it needs.

Partition	Wall limit	Hardware / best use
`htc`	4 h	Default for short CPU and GPU work; large accelerator pool.
`public`	7 days	Non-preemptable CPU/GPU jobs that need more than 4 h.
`general`	14 days	Privately-owned CPU/GPU nodes via `private` or `grp_*` QOS.
`lightwork`	1 day	Light development, compilation, bulk I/O, and `vscode`; max 8 cores.
`highmem`	7 days	Memory-heavy CPU work, up to 2 TB; normal `public` use may have a lower cap.
`arm`	7 days	ARM/aarch64 nodes with Grace Hopper (`gh200`) GPUs.
`fpga`	7 days	FPGA, Vector Engine, and special accelerator workloads.

ACCESS / QOS OVERLAYS

QOS	Wall cap	Use it for
`public`	Partition limit	Default, non-preemptable access to public resources.
`debug`	15 min	High-priority smoke tests; one running job and two submitted jobs per user.
`private`	Partition limit	Preemptible access to idle buy-in nodes; owners may cancel the job.
`grp_*`	Site/group limit	Your group's owned nodes, when your account provides the QOS.
`long`	14 days	Approved long batch jobs on `public` or `highmem`; not for interactive jobs.
`class`	1 day	Course accounts; per-user CPU, memory, GPU, and job-count caps.

DECISION / ROUTING IN ONE GLANCE

Routing: up to 4 h -> `htc`; up to 15 min and urgent -> `-p htc -q debug`; more than 4 h -> `public`; more than 4 h and preemption is acceptable -> `-p general -q private`. Validate unusual combinations with `sbatch --test-only`.

CLI / SOLX WORKFLOW

```
solx init                    # create ~/.config/solx/config.toml
solx config edit             # define job templates and [keep] paths
solx job start debug -n      # preview the salloc command
solx job start debug         # allocate; waits for the grant and prints the job ID
solx job jump                # open a shell on the compute node
solx job time                # show remaining wall-time
exit                         # leave the shell; the allocation keeps running
solx job stop                # cancel when done; prompts before acting
```

`job` also accepts `jobs`; `job list` accepts `ls`; and `solx jump` is an alias for `solx job jump`.

CLI / SOLX COMMANDS

Command	Purpose / important options
`solx init [-f]`	Write starter config; `-f` / `-y` overwrites.
`solx job list`	List your jobs (`squeue --me`).
`solx job start [TEMPLATE]`	Start an interactive allocation; `-n`, `--timeout`, and `--` passthrough.
`solx job jump [JOBID]`	Attach with `srun --pty`; `-q` hides nesting/selection notes.
`solx job time [JOBID]`	Print remaining time in `D-HH:MM:SS`.
`solx job stop [JOBID]`	Cancel a job; `-n` previews and `-y` skips confirmation.
`solx keep`	Renew flagged scratch files and directories selected by `[keep]`; see below.
`solx config show` / `edit`	Inspect resolved config or open it in `$EDITOR`.
`solx completions bash|zsh|fish`	Emit a static shell-completion script.
`solx cheatsheet`	Print this quick reference as text.
`solx version` / `help`	Aliases of `--version` / `--help`.

CLI / OUTPUT AND SAFETY

Situation	Rule
Human terminal	Data commands print aligned text.
Pipe or agent	Output auto-switches to JSON; `--json` forces it. Put `--json` before `job start`.
Destructive command	`job stop` and `keep` show a plan and prompt; `-n` previews, `-y` confirms.
Non-interactive session	A command that needs confirmation refuses instead of hanging.
Missing job ID	Inside an allocation, use `$SLURM_JOB_ID`; on login, `time` / `jump` pick the most recent job.
Multiple jobs on login	`stop` refuses to guess; pass the job ID.
Jump from another job	`solx job jump <JOBID>` safely targets that allocation and warns about nesting; use `-q` to silence the note.

```
solx --json job list | jq '.[].job_id'
solx job start gpu --timeout 20m -- --mem=128G    # last salloc flag wins
solx job stop 12345 -n                            # preview, never cancel
```

SCRATCH / SOLX KEEP

Only warning-CSV paths matched by `[keep]` are walked. Writable files and directories, including flagged roots and collaborator-owned entries, are renewed; symlinks are skipped and `/scratch` is never scanned blindly.

```
solx keep --dry-run -v       # inspect the full plan first
solx keep                    # execute with a confirmation prompt
solx --json keep -n          # machine-readable plan and capped path sample
solx --json keep -y          # execute; exact renewal and failure counts
```

Controls: `--stage all|inactive|over90|pending`, `--csv-dir DIR`, `-j N`, `-v`, `-n`, and `-y`. JSON reports `files_touched`, `dirs_touched`, and `failures` (`dirs` means matched roots); any failure exits 1. Run large renewals on the DTN, a compute node, or a short batch job - not on a throttled login node.

BATCH / SLURM BASICS

```
sbatch job.sh                # submit a batch script
squeue --me                  # your jobs (human alias: myjobs)
scancel <jobid>              # cancel
scontrol show job <jobid>    # full detail
sbatch --test-only job.sh    # validate without submitting
interactive                  # quick shell; defaults to htc/public, 1 core, 4 h
```

Minimal `#SBATCH` header (time format is `D-HH:MM:SS`):

```
#!/bin/bash
#SBATCH -p htc
#SBATCH -q public
#SBATCH -t 0-04:00:00
#SBATCH -c 8
#SBATCH --gres=gpu:a100:1
#SBATCH --mem=64G
#SBATCH -o slurm.%j.out
```

Start from `/packages/public/sol-sbatch-templates/templates/` when a supplied template fits. Use `solx job start` for interactive work and `sbatch` for unattended batch work.

TRANSLATE / SOLX AND SLURM

	Raw Slurm
`solx job start [TEMPLATE]`	`salloc` / `interactive` with template flags.
`solx job jump [JOBID]`	`srun --jobid=ID --overlap --pty $SHELL`.
`solx job list`	`squeue --me`.
`solx job time [JOBID]`	`squeue -h -j ID -o %L`.
`solx job stop -y ID`	`scancel ID`.

CHECKLIST / SAFE DEFAULTS

Rule	Default
Preview changes	Use `-n` / `--dry-run` before `job stop`, `keep`, or allocation changes.
Parse output	Use `solx --json ...` or native Slurm fields; do not scrape colorized wrappers.
Route short jobs	Use `htc` for work up to 4 h, including GPU jobs.
Protect the login node	Move compute and metadata-heavy I/O to a compute node, batch job, or DTN.
Preserve queue priority	Diagnose `PENDING`; update a viable route in place instead of resubmitting.

DIAGNOSE / PENDING JOBS

```
squeue --me -t PD -O "JobID,Reason:50,StartTime"   # reason and estimated start
scontrol show job <id>                             # all fields for one job
```

Reason	Response
`Priority` with low fairshare	Report the ETA and wait; resubmitting does not improve priority.
`ReqNodeNotAvail`	Check whether the requested node is reserved, drained, or down.
`Resources`	Right-size or reroute if another eligible partition starts sooner.

Only capacity-bound jobs benefit from rerouting. If a reroute helps, preserve the accrued queue priority with `scontrol update job <id> Partition=... QOS=...` instead of canceling and resubmitting.

AUTOMATION / STATUS COMMANDS

You want	Human command	Parse this
Fairshare / priority	`myfairshare`	`myfairshare` -> `RealFairShare`.
Scratch quota	-	`beegfs-ctl --getquota --uid $USER`.
Current jobs	`myjobs`	`squeue --me -O JobID,State,Reason`.
Pending start estimate	`thisjob ID`	`scontrol show job ID` -> `StartTime=`.
Finished-job efficiency	`seff ID`	`seff ID`.
Partition capacity	`showparts`	`sinfo -h -o "%P %a %l %D %t"`.
Free GPUs	`showgpus`	`sinfo -h -O "Partition,StateLong,Gres,GresUsed"`.

`my*` / `show*` tools are colorized for people. Agents should prefer native Slurm fields or `solx --json`; free GPUs equal `Gres` minus `GresUsed`.

CONNECT / REMOTE SERVICES

```
# On a Sol login node: register a VS Code tunnel on lightwork.
vscode

# On your laptop: forward Jupyter on compute node $NODE, port 8888.
ssh -N -L 8888:localhost:8888 -J $USER@login.sol.rc.asu.edu $USER@$NODE
```

Get `$NODE` from the `NODELIST` column in `squeue --me`. Bind services to `localhost`, never `0.0.0.0`, on shared nodes.

FILES / STORAGE AND I/O

Path	Use	Policy
`/scratch/$USER`	Datasets, caches, checkpoints, outputs.	Temporary, not backed up; inactive files are purged.
`/home/$USER`	Code, config, and small user installs.	Persistent, backed up, small quota.

```
export HF_HOME=/scratch/$USER/.cache/huggingface
export UV_CACHE_DIR=/scratch/$USER/.cache/uv
```

Use the DTN (`ssh soldtn`) for bulk transfer and metadata-heavy I/O. Use a compute node or batch job for compute. Keep heavy work off login nodes.